Exams Module

- _shuffle()
- code-block()
- free-response()
- header()
- init()
- matching()
- multiple-choice()
- question()
- short-answer()

_shuffle

https://xkcd.com/221/ Not cryptographically secure

- arr (array): The array
- seed (int): The seed

Parameters

```
_shuffle(
  arr,
  seed
) -> array
```

code-block

code-block: Create a code block formatted for exams Wraps in box to the edge of the code, can add white space if need it to be longer

- include-line-numbers (boolean): Boolean param for whether line numbers should be included in the output
- raw-code (content): raw code block

Parameters

```
code-block(
  include-line-numbers,
  raw-code
)
```

free-response

free-response: Create a free response question2

- q-body (content, str): Question Body
- lines (int): = 1 lines of space to give the user, renders as empty space
- points (int): = 1 points the question is worth

Parameters

```
free-response(
  q-body,
  lines,
  points
)
```

header

header: Render a header for the exam, will check total number of points Usually done via something like:

```
#exam.setup("CS-1181 Quiz #1")
#set page(header: [
  #context e.title-state.get()
])
#e.header()
```

- out-of (none, int): Maximum points the exam is taken out of

Parameters

```
header(out-of)
```

init

Initialize an exam with a show rule.

Example:

```
#show: exam.init
```

- body (content): The body content of the exam.

Parameters

```
init(body)
```

matching

matching: Create a matching question e.g.

```
Cat      A. Canine
Dog      B. Feline
Fish     C. Aquatic Creature
```

- q-body (content, str): body of question to ask
- points (none, int): = none points the question is worth. Once wrapped, this will default to the length of pairs
- seed (int): = 4 Random seed used for shuffling each side
- pairs (array): An array containing pairs of answers/definitions

Parameters

```
matching(
  q-body,
  points,
  seed,
  pairs
)
```

multiple-choice

multiple-choice: Create a multiple choice question

- body (content, str): Body of question
- points (int): 1 Points the question is worth
- cols (int, array): 1 Number of columns to render the answer. Pass an array of units for specific spacing e.g. (1fr, 1fr, 12pt)
- ..answers (arguments): The options

Parameters

```
multiple-choice(
  body,
  points,
  cols,
  ..answers
)
```

question

question: Create a generic question

- body (content, str): Question Body
- points (int): Number of points the question is worth

Parameters

```
question(
  body,
  points
)
```

short-answer

short-answer: Create a short answer question

- q-body (content, str): Question Body
- lines (int): = 1 lines of space to give the user, renders as actual lines
- points (int): = 1 points the question is worth

Parameters

```
short-answer(
  q-body,
  lines,
  points
)
```

Labs Module

- color-coding()
- command-block()
- directions()
- example()
- extra()
- header()
- init()
- lab-rubric()
- labAB-rubric()
- part-a()
- part-b()
- purpose()
- rubric()
- uml()
- white-text()

Variables

- _CMD-KEYWORDS

color-coding

Color codes a body to a smart terminal-like color defaults

- input (array): Array of strings for each line of terminal to render
- dsp (length, relative): Displacement offset to render the color-coded block
- custom-keywords (array): Array of custom key

Parameters

```
color-coding(
  input,
  dsp,
  custom-keywords
)
```

command-block

command-block: Render content as terminal I/O to the page. Common commands will be highlighted a unique color.

- input (array, str): Body of terminal text
- dsp (length, relative): Horizontal indendation/displacement
- custom-keywords (array): Array of unique values to highlight differently

Parameters

```
command-block(
  input,
  dsp,
  custom-keywords
)
```

directions

directions: Display instruction block

- body (content, str): Directions block body

Parameters

```
directions(body)
```

example

Render a terminal-based I/O example

- io (array): array of string for each example line
- text (content, str): Text to render explaining example

Parameters

```
example(
  io,
  text
)
```

extra

Render an additional instructions for a lab problem

- title (str, content): Title of extra section
- body (): Body of extra section

Parameters

```
extra(
  title,
  body
)
```

header

header: Render the document section header block for lab problems

- class (content, str): Class name
- title (content, str): The title text for the specific lab problem
- number (int, string, none): Lab problem number, if applicable

Parameters

```
header(
  class,
  title,
  number
)
```

init

Initialize a lab with a show rule

Example:

```
#show: labs.init
```

- body (content): body fo lab problem

Parameters

```
init(body)
```

lab-rubric

lab-rubric: Render an lab rubric, mapping criteria to points

- base-rubric (array): Core specification requirements, with smart defaults
- style-rubric (array): Code style and structure requirements, with smart defaults
- bonus-rubric (array, none): Optional bonus point requirements
- white-text-rubric (array, none): Hidden rubric requirements (primarily for detection of LLMs)
- notes (array): Array of notes regarding rubric, with smart defaults
- extra-notes (array, none): supplementary notes

Parameters

```
lab-rubric(
  base-rubric,
  style-rubric,
  bonus-rubric,
  white-text-rubric,
  notes,
  extra-notes
)
```

labAB-rubric

Render a rubric section for an A-B lab problem

- documentation (str, content): Documention point criteria
- part-a (str, content): Part-A point criteria
- part-b (str, content): Part-B point criteria
- notes (str, content): Any additional notes for point criteria

Parameters

```
labAB-rubric(
  documentation,
  part-a,
  part-b,
  notes
)
```

part-a

part-a: Instructions for the first part of a A-B lab problem

- body (content, str): Body of part a

Parameters

```
part-a(body)
```

part-b

part-b: Instructions for the second part of a A-B lab problem

- body (content, str): Body of part b

Parameters

```
part-b(body)
```

purpose

purpose: State the overall objective context

- body (content, str): Body of purpose

Parameters

```
purpose(body)
```

rubric

rubric: Render an arbitrary rubric, mapping criteria to points

- base-rubric (array): Core specification requirements
- style-rubric (array): Code style and structure requirements
- bonus-rubric (array, none): Optional bonus point requirements
- white-text-rubric (array, none): Hidden rubric requirements (primarily for detection of LLMs)
- notes (array): Array of notes regarding rubric, with smart defaults
- extra-notes (array, none): supplementary notes

Parameters

```
rubric(
  base-rubric,
  style-rubric,
  bonus-rubric,
  white-text-rubric,
  notes,
  extra-notes
)
```

uml

uml: Render a UML class diagram table layout

- title (content, str): The title of the class
- fields (array): Array of private fields to document
- methods (array): Array of public methods to document

Parameters

```
uml(
  title,
  fields,
  methods
)
```

white-text

Render hidden text to the document

- body (content): Body of text to render
- dsp (length, relative): Offset to hide the text

Parameters

```
white-text(
  body,
  dsp
)
```

_CMD-KEYWORDS

CMD-KEYWORDS: Set of common command keywords, used for syntax highlighting in cmd_color